

Question #1

1. Schema vs Instance

Schema is the structure/blueprint of the database (table names, columns, data types, constraints) that rarely changes. An instance is the actual data stored in the database at a particular point in time.

2. Foreign Key allows duplicates; Primary Key does not

A Primary Key uniquely identifies each row, so duplicates violate that uniqueness constraint. A Foreign Key references a PK in another table — multiple rows can reference the same parent record (e.g., multiple orders by the same customer), so duplicates are permitted.

3. "Max 21 credit hours" is a Semantic/Application-Based Constraint

Relational constraints (PK, FK, NOT NULL) are enforced directly by the DBMS engine. The 21-credit-hour rule requires checking the sum of enrolled credits per student per semester — this cannot be expressed as a simple column/table constraint and must be enforced via application code or triggers.

4. Why 2PL Guarantees Conflict Serializability

In 2PL, once a transaction releases any lock (shrinking phase begins), it cannot acquire new locks. This ensures the lock acquisition order among transactions creates a consistent partial order equivalent to some serial execution — preventing cyclic conflicts in the precedence graph.

5. Does the given scenario violate 2PL? Yes, it violates 2PL.

In 2PL, once a transaction enters the shrinking phase (releases a lock on Data Item A), it cannot acquire new locks. Here, the transaction releases the lock on A and then requests a lock on B in the same execution — it re-enters the growing phase after shrinking. This is a direct violation of the shrinking phase rule of 2PL.

Question #2

Query 1: All farmers with crops (LEFT JOIN)

```
SELECT F.full_name, F.city, C.crop_name, C.variety, C.quantity_kg,  
       C.base_price  
FROM FARMER F  
LEFT JOIN CROP C ON F.farmer_id = C.farmer_id;
```

Query 2: Winning bids with buyer, crop and auction details

```
SELECT B.full_name, B.buyer_type, C.crop_name, A.location,  
       BI.bid_amount
```

```

FROM BID BI
JOIN BUYER B ON BI.buyer_id = B.buyer_id JOIN AUCTION
A ON BI.auction_id = A.auction_id JOIN CROP C ON
A.crop_id = C.crop_id WHERE BI.bid_status = 'Won';

```

Query 3: Auctions with more bids than average

```

SELECT A.auction_id, A.location, A.status FROM
AUCTION A
JOIN BID BI ON A.auction_id = BI.auction_id GROUP BY
A.auction_id, A.location, A.status HAVING
COUNT(BI.bid_id) > (
    SELECT AVG(bid_count)
    FROM (SELECT COUNT(bid_id) AS bid_count
          FROM BID GROUP BY auction_id) AS avg_table
);

```

Query 4: Buyers with total winning bid > city average

```

SELECT B.full_name, B.city FROM
BUYER B
JOIN BID BI ON B.buyer_id = BI.buyer_id WHERE
BI.bid_status = 'Won'
GROUP BY B.buyer_id, B.full_name, B.city HAVING
SUM(BI.bid_amount) > (
    SELECT AVG(city_total) FROM (
        SELECT B2.buyer_id, SUM(BI2.bid_amount) AS city_total FROM BUYER
        B2
        JOIN BID BI2 ON B2.buyer_id = BI2.buyer_id
        WHERE BI2.bid_status = 'Won' AND B2.city = B.city GROUP BY
        B2.buyer_id
    ) AS city_avg
);

```

Query 5: Farmer with highest base_price crop (subquery)

```

SELECT F.full_name, F.city, F.bank_account FROM
FARMER F
JOIN CROP C ON F.farmer_id = C.farmer_id
WHERE C.base_price = (SELECT MAX(base_price) FROM CROP);

```

Query 6: Per city — distinct crops and revenue (>5 crops)

```

SELECT F.city,
    COUNT(DISTINCT C.crop_id) AS total_crops,
    SUM(BI.bid_amount) AS total_revenue
FROM FARMER F
JOIN CROP C ON F.farmer_id = C.farmer_id JOIN
AUCTION A ON C.crop_id = A.crop_id JOIN
BID BI ON A.auction_id = BI.auction_id WHERE
BI.bid_status = 'Won'
GROUP BY F.city
HAVING COUNT(DISTINCT C.crop_id) > 5
ORDER BY total_revenue DESC;

```

Query 7: Payments for winning bids in Completed auctions

```

SELECT P.payment_id, P.amount_paid, P.pay_method, P.pay_date FROM
PAYMENT P
JOIN BID BI ON P.bid_id = BI.bid_id

```

```

JOIN AUCTION A ON BI.auction_id = A.auction_id WHERE
BI.bid_status = 'Won'
AND A.status = 'Completed';

```

Query 8: Per crop — avg/max/min winning bid (avg > 50,000)

```

SELECT C.crop_name,
       AVG(BI.bid_amount) AS avg_bid,
       MAX(BI.bid_amount) AS max_bid,
       MIN(BI.bid_amount) AS min_bid
FROM CROP C
JOIN AUCTION A ON C.crop_id = A.crop_id JOIN
BID BI ON A.auction_id = BI.auction_id WHERE
BI.bid_status = 'Won'
GROUP BY C.crop_name
HAVING AVG(BI.bid_amount) > 50000 ORDER
BY avg_bid DESC;

```

Query 9: Buyers from Active/Completed auction cities (IN subquery)

```

SELECT full_name, city, buyer_type FROM
BUYER
WHERE city IN (
    SELECT location FROM AUCTION
    WHERE status IN ('Active', 'Completed')
);

```

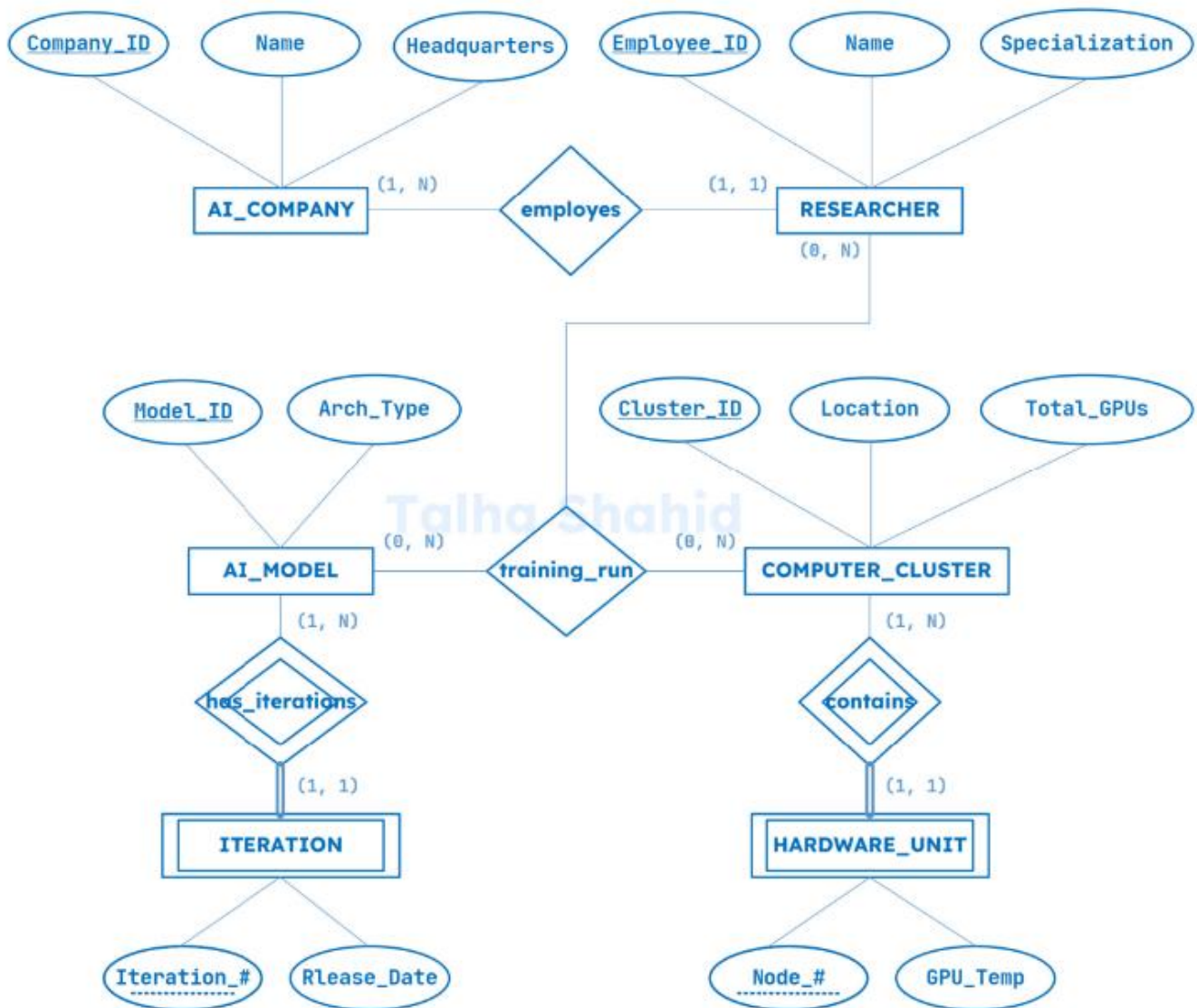
Query 10: Per buyer — auctions, bids, revenue, avg bid (>=3 auctions, revenue >100,000)

```

SELECT B.full_name,
       COUNT(DISTINCT BI.auction_id) AS total_auctions,
       COUNT(BI.bid_id) AS total_bids,
       SUM(BI.bid_amount) AS total_revenue,
       AVG(BI.bid_amount) AS avg_bid
FROM BUYER B
JOIN BID BI ON B.buyer_id = BI.buyer_id JOIN
AUCTION A ON BI.auction_id = A.auction_id GROUP BY
B.buyer_id, B.full_name
HAVING COUNT(DISTINCT BI.auction_id) >= 3
AND SUM(BI.bid_amount) > 100000
ORDER BY total_revenue DESC;

```

Question #3



Question #4

a) First Normal Form (1NF)

- **Violating Attributes:** ST (Skill Type) and GF (Game Format) because they contain multi-valued, non-atomic data.
- **Resulting 1NF Table Structure:**

Player_1NF (PI, PN, PR, BTS, BWS, ST, GF, PA, TI, TN, TC)

b) Second Normal Form (2NF)

- **Violating Attributes:** PN, PR, BTS, BWS, PA, TI, TN, TC
 - *Reason: These non-prime attributes depend only on PI (part of the composite key), causing **Partial Dependencies**.*

Functional Dependencies (FDs) after 2NF:

1. $PI \rightarrow PN, PR, BTS, BWS, PA, TI, TN, TC$
2. Composite relations for multi-valued tracking: (PI, ST) and (PI, GF)

Resulting 2NF Table Structures:

- **Player_Details** (PI, PN, PR, BTS, BWS, PA, TI, TN, TC)
- **Player_Skills** (PI, ST)
- **Player_Formats** (PI, GF)

c) Third Normal Form (3NF)

- **Violating Attributes:** TN (Team Name) and TC (Team Coach)
 - *Reason: They depend on TI (Team ID), which depends on PI. This creates a **Transitive Dependency** ($PI \rightarrow TI \rightarrow TN, TC$)*

Final Functional Dependencies (FDs):

1. $PI \rightarrow PN, PR, BTS, BWS, PA, TI$
2. $TI \rightarrow TN, TC$

Final 3NF Table Structures (Final Answer):

1. **PLAYER** (PI, PN, PR, BTS, BWS, PA, TI)
2. **TEAM** (TI, TN, TC)
3. **PLAYER_SKILLS** (PI, ST)
4. **PLAYER_FORMATS** (PI, GF)

Question #5

RA 1: Passengers on flights with ticketPrice > 150,000 AND totalFare > 200,000

```
pi(passengerName, departureCity, arrivalCity) (sigma(ticketPrice >
150000 AND totalFare > 200000)
(Passenger |><| Booking |><| Flight))
```

RA 2: Distinct aircraft models on delayed flights with ticketPrice > 100,000

```
delta(pi(model)
(sigma(status = 'Delayed' AND ticketPrice > 100000) (Flight |><|
AircraftAssignment |><| Aircraft)))
```

RA 3: Airline names with count of pilots earning above average salary

```
airlineName G COUNT(pilotID)
(sigma(salary > AVG_salary)(Pilot |><| Airline))
```

where AVG_salary = pi(AVG(salary))(Pilot) -- computed separately

RA 4: Passenger names and total spending on international flights > 500,000

```
passengerID G SUM(totalFare) (sigma(departureCity !=
arrivalCity)
(Booking |><| Flight |><| Passenger))
-- then filter SUM(totalFare) > 500000
```

RA 5: Aircraft models NEVER assigned to active flights (Set Difference)

```
pi(model)(Aircraft) MINUS
pi(model)(sigma(status = 'Active')(Flight) |><| AircraftAssignment |><| Aircraft)
```

Question # 6

1. Is the schedule Recoverable? NO —

Not Recoverable.

T2 reads X after T1 writes it (r2(X) follows w1(X)), creating a read dependency: T2 reads-from T1. For recoverability, T2 must not commit before T1 commits. However, T2 commits (c2) and then T1 aborts (a1). T2 has committed based on a dirty value — this is unrecoverable.

2. Does the schedule lead to Cascading Rollback?

YES — Cascading rollback exists in the structure.

T2 read uncommitted data written by T1 (r2(X) before T1 committed). If T1 had aborted before T2 committed, T2 would need to be rolled back as a consequence — that is cascading rollback. In this schedule, since T2 committed first, it becomes unrecoverable rather than a cascade.

3. Is the schedule Strict? NO —

Not Strict.

A strict schedule requires that no transaction read or write a data item X written by an uncommitted transaction. Here, T2 reads X (r2(X)) after T1 writes X (w1(X)) but before T1 commits — this violates the strictness condition.

4. Modified Schedule — Make it Recoverable

r1(X), w1(X), c1, r2(X), w2(X), c2

-- T2 only reads X after T1 commits => recoverable + cascadeless

Question #7

Part a — Concurrency Issues

Problem	Explanation
Dirty Read	A transaction reads data written by another uncommitted transaction; if that transaction aborts, the read data is invalid.
Unrepeatable Read	A transaction reads the same data item twice and gets different values because another transaction modified and committed it in between.
Lost Update	Two transactions read and then write the same data item; one write overwrites the other, causing one update to be permanently lost.
Phantom Read	A transaction re-executes a query and finds new rows inserted by another committed transaction, changing the result set.
Incorrect Summary	An aggregate (SUM/COUNT) is computed while another transaction is concurrently updating the data, yielding a wrong total.

Part b — Schedule Trace

Initial values: balx = 20, baly = 30, balz = 50, sum = 0

Time	T1	T2	balx	baly	balz	sum
t1		begin_transaction	20	30	50	0
t2	begin_transaction	sum = 0	20	30	50	0
t3	read(balx)=20	read(balx)=20	20	30	50	0
t4	balx = 20-10 = 10	sum = 0+20 = 20	20	30	50	20
t5	write(balx)=10	read(baly)=30	10	30	50	20
t6	read(balz)=50	sum = 20+30 = 50	10	30	50	50
t7	balz = 50+10 = 60		10	30	50	50
t8	write(balz)=60		10	30	60	50
t9	commit	read(balz)=60	10	30	60	50
t10		sum = 50+60 = 110	10	30	60	110
t11		commit	10	30	60	110

Concurrency Issue Identified: INCORRECT SUMMARY Issue occurs at: t3

T2 read balx = 20 at t3 (before T1's update). T1 then updated balx to 10 at t5. The correct sum should be: 10 + 30 + 60 = 100, but T2 computed 110. T2 read the old (stale) value of balx because T1 was concurrently modifying it — this is an Incorrect Summary (a form of Unrepeatable Read in aggregate context).

Question #8

Q/Part No. Page No. 13 Rough Work

Q8a)

S1			S2		
T1	T2	T3	T1	T2	T3
$\lambda_1(x)$			$\lambda_1(x)$		
	$\lambda_2(z)$			$\lambda_2(z)$	
$\lambda_1(z)$		$\lambda_3(x)$			$\lambda_3(x)$
		$\lambda_3(x)$	$\lambda_1(z)$		
		$\lambda_3(y)$		$\lambda_2(y)$	
$w_1(x)$					$\lambda_3(y)$
		$w_3(y)$	$w_1(x)$		
	$\lambda_2(y)$			$w_2(z)$	
	$w_2(z)$				$w_3(y)$
	$w_2(y)$			$w_2(y)$	

b) for S1:

```

graph LR
    T1((T1)) --> T2((T2))
    T3((T3)) --> T1
    T3 --> T2
  
```

no loop/cycle exists, hence conflict serializable

equivalent serial schedule: $T_3 \rightarrow T_2 \rightarrow T_1$ ~~$T_3 \rightarrow T_1 \rightarrow T_2$~~

remove nodes with indegree = 0

for S2:

```

graph LR
    T1((T1)) --> T2((T2))
    T2 --> T3((T3))
    T3 --> T1
  
```

cycle/loop exists, hence not conflict serializable.